Public editions adapted from real Career Agent Pro outputs

Edição pública adaptada de uma saída real do produto. Material educacional gerado por IA e revisado para publicação. Todos os contextos de arquitetura, equipes, incidentes, requisitos e avaliação são hipóteses didáticas de plataformas fictícias. Não descrevem informações internas confirmadas, questões oficiais, contratação ou estratégia de empresas reais. Sem vínculo ou endosso de Prezensa, Globo, WhatsApp ou Meta.

System design Android: streaming e feed offline

Engenharia Android para plataforma fictícia de streaming. Nível didático: sênior. Este documento não descreve uma vaga real.

Visão Geral

Este documento apresenta dois desafios de System Design propostos para praticar decisões em nível sênior. Cada desafio inclui contexto, requisitos, armadilhas ocultas, perguntas de simulação e os trade-offs que você deve dominar.

> **Dica estratégica:** Em entrevistas de System Design, o diferencial não está em acertar a “resposta certa” - está em demonstrar como você pensa, quais problemas você enxerga antes de ser perguntado e como você justifica suas escolhas.

Desafio 1 - Sistema de Gerenciamento de Estado do Player de Streaming

Contexto

Considere uma plataforma fictícia de streaming. O player deve lidar com bufferização, pausa, erros, transições de ciclo de vida, controles de mídia e seleção adaptativa de qualidade. A escala deve ser definida com dados sintéticos do exercício.

Enunciado fictício para discussão:

> Enunciado fictício: uma implementação legada combina RxJava e callbacks ligados à Activity e apresenta falhas durante transições para background. Como investigar e refatorar estado, lifetimes e efeitos sem reescrever todo o player?

Requisitos Funcionais

- Suporte aos estados: `Idle` · `Loading` · `Buffering` · `Playing` · `Paused` · `Error` · `Ended`
- Estado deve **sobreviver à rotação de tela** sem reiniciar o vídeo
- Em background, o áudio deve continuar (se habilitado) com notificação de controle de mídia
- Ao retornar para foreground, a UI deve re-sincronizar com o estado atual **sem flickering**
- Erros de rede devem acionar retry automático com **backoff exponencial**
- A qualidade do stream (HLS/DASH) deve ser trocada adaptativamente **sem interromper a reprodução**

Requisitos Não Funcionais

Critério	Meta
Latência de atualização da UI após mudança de estado	< 16ms (1 frame)
Memory leaks	Zero - nenhuma referência viva para Activity ou View
Compatibilidade	Android 8 (API 26) até Android 14
Testabilidade	Lógica de estado 100% testável sem dispositivo físico
Manutenibilidade	Suporte a 5 desenvolvedores trabalhando em paralelo

---

Desafios Ocultos

> Aprofundamento: investigue estes riscos adicionais e explicite as hipóteses antes de escolher uma solução. A rubrica serve para orientar estudo, não classificar profissionais.

1. O problema do "estado fantasma" Quando o processo é morto pelo sistema operacional (low memory killer), o estado do player é destruído. Na próxima abertura, o app precisa decidir: retomar do ponto exato ou reiniciar? Essa decisão não está nos requisitos - você deve perguntar ou propor uma abordagem antes de continuar.

2. Concorrência e application Looper: ExoPlayer deve ser acessado pelo application thread configurado; callbacks de Player.Listener são entregues nesse Looper. Threads de playback internas não são licença para acessar o player de qualquer thread. Adapte tarefas externas ao dispatcher correto e mantenha I/O bloqueante fora da Main.

3. Contextos e lifetimes: referências de longa duração a Activity/View podem vazar. Escolha contexto, remova listeners e libere recursos conforme o proprietário. MediaSessionService é uma opção para reprodução em background; sua ausência não implica automaticamente um leak.

4. Ownership de reprodução: o ViewModel pode ser limpo enquanto um serviço permanece ativo, mas nenhum deles sobrevive como instância à morte do processo. Defina um proprietário para o player, comunicação com UI e recuperação via dados persistidos; não use um escopo de tela para toda a vida da reprodução.

5. Frequência e estado: não assuma callback periódico de posição em Player.Listener. Se a UI precisar progresso contínuo, consulte a posição na cadência justificada e no application thread. Use distinctUntilChanged/sample quando medições indicarem custo; não descarte transições críticas de playback por debounce indiscriminado.

Perguntas para simulação

Abertura

> *“Por onde você começa? Qual é o primeiro problema que você identifica aqui?”*

Sobre estado e ciclo de vida do processo

> *“Você mencionou ViewModel para gerenciar o estado. O que acontece quando o processo é morto pelo SO? Como o usuário experimenta isso?”*

Sobre concorrência

> *“O ExoPlayer emite eventos como `onPlaybackStateChanged` em qual thread? Como você garante que isso não vai causar ANR ou race condition?”*

Sobre background

> *“Se o Android matar o processo para liberar memória enquanto o áudio toca em background, o que acontece com a notificação de controle de mídia? Quem é responsável por ela?”*

Sobre testabilidade

> *“Como você testaria a lógica de retry com backoff exponencial sem simular uma rede real?”*

Sobre escolha de primitiva

> *“Você escolheria `StateFlow` ou `SharedFlow` para expor os eventos do player para a UI? Me explica o raciocínio.”*

Sobre escala de time

> *“Cinco desenvolvedores vão trabalhar nesse módulo em paralelo. O que você colocaria no design para evitar conflitos e facilitar code review?”*

Alternativas e trade-offs

Decisão	Opção A	Opção B	O que demonstra maturidade
Gerenciamento de estado	`StateFlow` no ViewModel	`MutableLiveData` + Observer	Defender `StateFlow` para novos módulos, reconhecendo que `LiveData` ainda existe em código legado

Decisão	Opção A	Opção B	O que demonstra maturidade
Player em background	`MediaSessionService` (Media3 / Foreground Service)	ViewModel controlando ExoPlayer diretamente	Identificar que ViewModel não é suficiente para background de longa duração
Eventos do player → UI	Throttle/sample com operadores de Flow	Emitir todos os eventos	Identificar o problema de back pressure **sem ser perguntado**
Retry de rede	Retry no repositório com `retryWhen` (Flow)	Retry na ViewModel	Retry é lógica de infraestrutura - não de apresentação
Persistência de posição	Room (banco local)	SharedPreferences	Room para posição de reprodução + última mídia; justificar a escolha
Separação de responsabilidades	Módulo `:feature:player` isolado	Player acoplado à feature de catálogo	Propor modularização sem esperar a pergunta

Desafio 2 - Sistema Offline-First para Feed de Notícias

Contexto

Considere o módulo fictício de notícias da mesma plataforma. Ele deve permitir leitura offline e atualização controlada. Volume de instalações, tamanho do catálogo e taxa de alteração são parâmetros sintéticos a declarar antes de dimensionar cache e paginação.

Enunciado fictício para discussão:

> Enunciado fictício: desenhe um feed que permita leitura offline, sincronize quando houver conexão e exponha claramente idade e origem dos dados. Como definir frescor, resolução de conflitos e política de retenção?

Requisitos Funcionais

- Exibir o feed **mesmo sem conexão** à internet
- Ao reconectar, atualizar o feed automaticamente - sem o usuário precisar puxar para atualizar
- Suporte a **paginação infinita**
- Mensagens FCM sinalizam possível conteúdo novo; sincronizar quando condições do SO e rede permitirem e também ao abrir o app. Push não é garantia de frescor.
- Marcar notícias como lidas - persistir localmente e sincronizar com o servidor quando houver conexão
- Conteúdo com mais de 24 horas deve ser sinalizado visualmente como **desatualizado**

Requisitos Não Funcionais

Critério	Meta
Cold start com feed visível	< 300ms (mesmo offline)
Impacto da sync em background na UI	< 5% de CPU adicional
Crescimento do banco local	Controlado - policy de retenção obrigatória
Suporte a múltiplas contas	Dados isolados por perfil
Compatibilidade	Android 6+ (API 23)

Desafios Ocultos

> **Atenção:** Identificar esses pontos sem ser perguntado é o diferencial esperado de um candidato sênior.

1. Paginação e consistência: Paging 3 + Room + RemoteMediator é uma opção. Se o servidor remover itens ou alterar a ordenação durante leitura offline, páginas por offset podem produzir lacunas e duplicação. Compare cursores estáveis, refresh de chaves remotas e invalidação atômica; mantenha ordem e regras de cache testáveis.

2. Conflito de dados em "marcar como lido" O usuário marcou uma notícia como lida offline. Ao sincronizar, o servidor indica que ela não foi lida. Qual versão vence? É necessário propor uma estratégia de resolução de conflito - timestamp, last-write-wins ou CRDT conceitual.

3. Coordenação: uma mensagem FCM pode sinalizar invalidação quando entregue; não garante execução imediata nem sincronização completa. WorkManager executa sob constraints. Centralize requests, deduplique sinais e preserve transações de cache para evitar refreshes concorrentes.

4. Isolamento de dados entre múltiplas contas Se dois usuários compartilham o mesmo dispositivo, o Room precisa particionar os dados por `userId`. Uma tabela sem essa separação é uma falha de design que o facilitador da simulação vai explorar.

5. Relógio do dispositivo vs. timestamp do servidor O requisito de sinalizar conteúdo desatualizado parece simples - mas o timestamp de criação vem do servidor, e o relógio do dispositivo pode estar incorreto. É necessário trabalhar com o tempo do servidor como referência.

Perguntas para simulação

Abertura

> *"Você mencionou Room. Quem aciona a sincronização quando o usuário abre o app? A ViewModel? O Repository? Um Service?"*

Sobre arquitetura

> *"Me desenha as camadas: quem chama quem? Onde vive a lógica de 'retorno do cache enquanto busco do servidor'?"*

Sobre paginação

> *“Você usaria Paging 3? O que o RemoteMediator faz exatamente? O que acontece quando o servidor retorna dados em uma ordem diferente do cache?”*

Sobre conflito de dados

> *“O usuário marcou a notícia como lida offline. Quando sincroniza, como você resolve o conflito?”*

Sobre background

> *“WorkManager e FCM podem disparar ao mesmo tempo. Como você evita chamadas duplicadas?”*

Sobre armazenamento

> *“Você escolheria Room ou um cache em memória puro para o feed? Por quê?”*

Sobre retenção de dados

> *“Quem decide quando deletar notícias antigas? Como você implementa isso sem bloquear a UI?”*

Alternativas e trade-offs

Decisão	Opção A	Opção B	O que demonstra maturidade
Estratégia de cache	Room como source of truth + API como fonte de atualização	In-memory cache + API direto	Persistência durável é necessária para este requisito; Room é uma opção, não a única. Justificar modelo, transações e migrações.
Paginação	Paging 3 + RemoteMediator	Paginação manual com offset	Paging 3 - com domínio dos trade-offs do RemoteMediator
Trigger de sincronização	WorkManager periódico + FCM push	Pull-to-refresh manual	Ambos coordenados - propor coordinator para evitar duplicidade
Resolução de conflito	Last-write-wins com timestamp	Servidor sempre vence	Depende do dado: para “lidos”, last-write-wins é razoável; para conteúdo editorial, servidor vence

Decisão	Opção A	Opção B	O que demonstra maturidade
Isolamento multi-conta	Tabelas com coluna `userId` + queries filtradas	Banco separado por conta	Comparar tabelas particionadas por conta com banco por conta; considerar isolamento, limpeza, migrações, número de contas e risco de queries sem filtro.
Retenção de dados	WorkManager agendado com policy configurável	Nunca deletar	WorkManager - operação de I/O fora da Main thread, policy explícita e configurável
Cold start < 300ms	Room retorna cache imediato; sync acontece em paralelo	Aguarda resposta do servidor	Cache first, sync async - nomear como **stale-while-revalidate** demonstra senioridade

Guia de Autoavaliação - Pleno vs. Sênior

Use esta tabela para identificar onde você está e o que precisa desenvolver antes da entrevista.

Comportamento	Nível Pleno	Nível Sênior
Identificação de problemas	Responde o que foi perguntado	Identifica problemas que não foram perguntados
Nível de abstração	Fala em classes e métodos	Fala em contratos, responsabilidades e invariantes
Trade-offs	Escolhe uma solução	Compara opções com critérios explícitos
Ciclo de vida	Conhece o básico de Activity/Fragment	Raciocina sobre o que acontece quando o processo morre
Testabilidade	Menciona testes no final	O design já nasce pensando em testabilidade
Escala de time	Pensa no próprio código	Considera como outros devs vão trabalhar no mesmo módulo
Postura na entrevista	Responde perguntas	Faz perguntas de clarificação antes de propor soluções

Pontos de atenção na autoavaliação

> **Atenção:** Nesta rubrica didática, estes comportamentos indicam lacunas a investigar e discutir; não predizem resultados em seleções reais.

- Propor `GlobalScope` para qualquer operação assíncrona
- Não mencionar ciclo de vida em nenhum momento dos desafios
- Aceitar requisitos ambíguos sem questionar
- Propor solução sem mencionar testabilidade
- Não saber explicar a diferença entre `StateFlow` e `SharedFlow` quando perguntado diretamente
- Tratar a ViewModel como o lugar correto para lógica de persistência ou chamadas de rede diretas

Limites e evidências da implementação

Separe estado de UI, recursos do player e dados persistidos. ViewModel ajuda em mudanças de configuração, mas não substitui restauração após morte do processo. Serviços também podem ser encerrados; definir recuperação e persistir apenas o necessário. No feed, diferencie cache vazio, dado antigo, atualização em andamento e falha de atualização.

Compare arquiteturas por invariantes e evidências: testes com scheduler controlado, cancelamento, callbacks após destruição da view, alteração de conta, relógio incorreto e falhas parciais. Metas numéricas são parâmetros do exercício; medir em uma matriz sintética de dispositivos e cargas antes de afirmar que uma alternativa cumpre o orçamento.

Referências técnicas e escopo

Referências de APIs públicas para estudo; não são evidência sobre arquitetura ou processos de empresas reais. Os contratos Kotlin são exemplos educacionais, sem implementação de produtos privados.

- Android: coroutines e ciclo de vida
- Android: WorkRequests e constraints
- Media3: ExoPlayer e application thread

Career Agent PRO

Edição pública adaptada de saída real. Profundidade técnica, hipóteses explícitas e decisões verificáveis.

www.career-agent-pro.com
support@main.career-agent-pro.com

© 2026 Career Agent PRO. Todos os direitos reservados.

ADAPTED PUBLIC EDITION